

Islamic University of Technology

Lab 5: Dynamic Programming Engineering

CSE 4810: Algorithm Engineering
ASH, ABT(PT)
July 9, 2026

General Instruction

This lab consists of **four independent tasks**. Unlike previous labs, written derivations are replaced by **live profiling, execution, and verbal defense**. For every task you must:

- Step 1.** Complete the `// TODO` sections in the provided C++ source file.
- Step 2.** Compile and run the program, feeding it the exact stress-test inputs specified.
- Step 3.** Produce the requested live artifact — a runtime plot, a memory log, or a comparison count — and keep your terminal/IDE open.
- Step 4.** Be ready to defend your engineering choices verbally using the artifact on screen; no written report is collected.

Files Provided

RodCutting.cpp Naive / Memoized / Bottom-Up rod-cutting skeleton
plotter.py Plots runtime/memory data (auto-detects by filename)

Task 1: The Performance Paradox in Rod Cutting

Problem: Given a rod of length n and a table of prices for smaller integer lengths, the goal is to determine the maximum revenue obtainable by cutting and selling the rod in various configurations. **Objective:** Visualize exponential vs. polynomial growth in real time and demonstrate how the call stack impacts physical memory.

Variant	Strategy	Why It Matters
<code>cut_rod_naive</code>	Pure recursion, tries every cut position with no reuse	Recomputes identical subproblems exponentially. Correct but scales at $O(2^n)$.
<code>cut_rod_memoized</code>	Top-down recursion with a memo array caching solved subproblems	Eliminates recomputation but still pays call-stack overhead per subproblem.
<code>cut_rod_bottom_up</code>	Iterative tabulation, builds solutions from length 0 upward	Removes the call stack entirely , trading recursion depth for a flat loop.

0.1 TODO 1: Implement `cut_rod_naive(n, prices)`

0.2 TODO 2: Implement `cut_rod_memoized(n, prices, memo)`

0.3 TODO 3: Implement `cut_rod_bottom_up(n, prices)`

Live Experimentation Setup

Runs to Perform

- **The Execution Run:** Run all three variants for rod lengths $n = 10, 20, 30$.
- **The Stress Test:** For $n = 40$, execute only the memoized and bottom-up versions. Kill the naive version if it has not returned within 30 seconds.
- **The Visualization:** Plot execution time vs. n for all three variants. Identify the exact point where the naive curve shoots upward vertically while the DP curves remain flat.

Compile & Run

```
1 g++ -O2 -o rodcut RodCutting.cpp
2 ./rodcut
3 python3 plotter.py data_rodcut.txt
```

Core Engineering Defense (Verbal Check)

Open your IDE and task manager/terminal and be ready to explain:

- **The Curve Break:** Point to your scatterplot and explain *why* the naive approach scales at $O(2^n)$ while both DP variants scale at $\Theta(n^2)$.
- **Stack Overflow Demonstration:** Force a stack overflow using the top-down memoized version by running a massively large n (e.g., $n = 50,000$). Explain what limits the OS places on recursion depth.
- **The Tabulation Advantage:** Show why the bottom-up version handles $n = 50,000$ flawlessly without crashing, linking this to the physical elimination of call-stack frames.

The instructor may ask you to increase n for the memoized version until it crashes on your machine, then report the exact n at which the crash occurred and explain why that number is machine-dependent rather than a fixed language constant.